Comparing Three Minimum Cut Algorithms: Stoer-Wagner, Karger, and Karger's Tree Packing

Programming Assignment 2: Group Algorithm Comparison (Theme 2: Deterministic vs Randomized)

Group member	Primary algorithm
Aarav Khemani	Stoer-Wagner (deterministic)
Rodrigo Uzal Sequeros	Karger's contraction algorithm (randomized contraction)
Rentaro Beattie	Karger's tree packing, "Minimum Cuts in Near-Linear Time" (randomized sampling)

Repository: https://github.com/Nousand/Min-Cut-Advanced-Algorithms-AS3
Video: https://youtu.be/49uuL96k92g

---

1. The Problem and the Algorithms

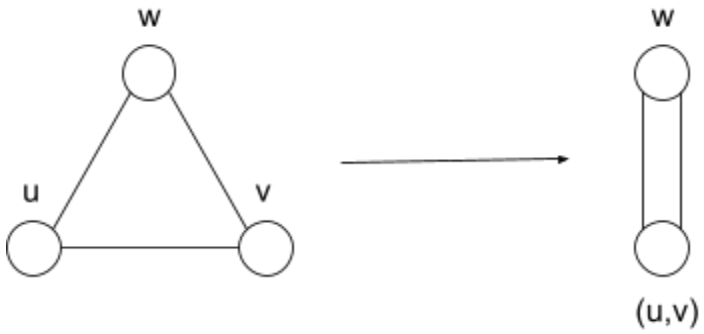
1.1 Problem statement

The problem: Given a connected, undirected, unweighted graph G = (V, E) with n vertices and m edges, find the smallest number of edges whose removal disconnects G (the global minimum cut). Input format used by every implementation: vertices numbered 0 to n−1, edges as a list of vertex pairs with each undirected edge listed once, no self-loops. Each algorithm returns the size of the minimum cut.

Why does it matter?: the minimum cut isolates the weakest link or capacity constraint in a network, which directly correlates with max-flow problems. According to the max-flow min-cut theorem, the maximum amount of flow in a network is identical to the capacity of the minimum s-t cut (Ford and Fulkerson, 1956), the variant of the problem where the cut must separate two given nodes s and t. Since the global minimum cut must separate a fixed node s from some other node t, it could be found with n − 1 max-flow computations, whereas Stoer and Wagner's algorithm "uses no flow techniques" (Stoer and Wagner, 1997).

The practical applications of this problem are many. In computer vision it is used for image segmentation and stereo vision (Boykov and Jolly, 2001; Roy and Cox, 1998), where a minimum s-t cut separates foreground objects from the background. In network reliability (Colbourn, 1987), a network becomes disconnected precisely when all the edges of some cut fail (Karger, 1999). Finally, in data clustering, the HCS algorithm repeatedly splits a graph along its minimum cut (Hartuv and Shamir, 2000).

Why does contraction make parallel edges appear even though the inputs are simple graphs?: Contracting two adjacent nodes u and v merges them into one super vertex {u, v}: the edges between them disappear (otherwise they would become self-loops) and all their other edges are preserved. If u and v share a common neighbour w, as in the figure below, the edges (u, w) and (v, w) now both join {u, v} to w, which is a parallel edge.



These parallel edges cannot simply be merged into one. Any cut that separates {u, v} from w must remove both of them, since each one is an edge of the original graph, so keeping only one would make that cut look smaller than it really is. Since Karger depends on contraction and Stoer–Wagner on merging vertices, both must handle parallel edges, and therefore also work on multigraphs. Karger keeps them as repeated entries in its edge list, while Stoer–Wagner stores the number of edges between each pair of vertices as a weight.

1.2 Stoer-Wagner (primary author: Aarav Khemani)

- **Key idea:** The algorithm runs n−1 phases. In each phase it builds a maximum adjacency ordering: start at any vertex and repeatedly add the vertex with the most edges into the set chosen so far. If s and t are the last two vertices added, the cut separating t from everything else (the "cut of the phase") is a minimum s–t cut (Stoer and Wagner, 1997). We record it, then merge s and t into one vertex, adding up their parallel edges. The global minimum cut either separates s and t, in which case it equals the cut of the phase, or it doesn't, in which case it survives in the merged graph and a later phase finds it. The smallest cut of the phase over all n−1 phases is the answer.
- **Complexity:** The matrix version takes $\Theta(n^3)$ time and $\Theta(n^2)$ memory: there are n−1 phases, and in each phase every vertex selection scans all n vertices and updates their keys. The priority queue version takes O(nm log n) time and $\Theta(n + m)$ memory: each phase pushes O(m) entries into a binary heap at O(log n) each. The original paper reaches $O(nm + n^2 \log n)$ with Fibonacci heaps (Stoer and Wagner, 1997).
- **Two data-structure variants of the same algorithm:** stoer_wagner (matrix, scan for maximum) and stoer_wagner_pq (adjacency maps with a lazy-deletion priority queue) Both versions return the same cut size on every instance used in the tests and benchmarks, and the benchmark stops if they ever disagree.
- **What makes it different:** Stoer-Wagner is deterministic and exact. It returns the true minimum cut on every input, does the same work every time on a given graph, and needs no repetitions, so there is no success probability to analyse. It uses neither randomness nor flow computations, and it handles parallel edges by design because it merges vertices. Its cost is paid up front in n−1 phases, whatever the graph looks like, and it serves as the reference answer for checking the randomized algorithms.

1.3 Karger's contraction algorithm (primary author: Rodrigo Uzal Sequeros)

- **Key idea:**Repeatedly pick a random edge and contract it, merging its two endpoints into one super-vertex while keeping any parallel edges, until only two super-vertices remain. The edges between them form a cut of the original graph. Since a minimum cut has few edges, a randomly chosen edge is unlikely to belong to it.
- **Complexity:** One run costs $O(m\ log\ n)$:$O(m)$ to shuffle the edges, plus a constant number of union–find operations per edge, each $O(log\ n)$with union by size. A single run preserves a fixed minimum cut with probability at least $2/(n(n\ -\ 1))$. Repeating it $t\ =\ 3n(n\ -\ 1)/2$ times and keeping the smallest cut found gives a failure probability of at most$e^{-3} \approx 0.05$, for a total running time of $O(n^2 m\ log\ n)$.
- **Implementation choices:** The edges are shuffled once per run instead of choosing a random edge at each step, which is equivalent. Union–find uses union by size without path compression. Contraction stops as soon as two groups remain, and then all m edges are scanned to count those crossing between the two groups. The number of runs t and the random seed are parameters, so every experiment can be reproduced.
- **What makes it different:** It is a Monte Carlo algorithm: its running time is fixed, but a single run can return a cut larger than the minimum, so the answer is only correct with high probability after repetition. It never underestimates, since every answer is a real cut. Compared with Stoer–Wagner, it is much shorter to implement but needs many repetitions.

1.4 Karger's tree packing (primary author: Rentaro Beattie)

- **Key idea:** The algorithm packs k spanning trees by iteratively running Kruskal's algorithm where edge weights equal their current load (how many previous trees use them), balancing usage to approximate a maximum tree packing.
  To break ties among equal-load edges, it shuffles edge indices uniformly at random before each Kruskal pass to produce a randomized minimum spanning tree rather than depending on a fixed original edge order. For every packed tree, it identifies the minimum cut by inspecting 1-respecting cuts and leveraging an algebraic recurrence with a bottom-up dynamic programming pass to evaluate 2-respecting cuts efficiently.
- **Complexity:** While the theoretical complexity of optimal tree-packing frameworks $O(m\ log^3 n)$ using Gabow's algorithm or fast dynamic tree data structures like Link-Cut Trees, the implemented complexity runs in $O(k\ \cdot\ m \log m)$ to shuffle and build k trees via Kruskal's algorithm. Evaluating 1-respecting cuts takes $O(n\ \cdot\ m)$ per tree, while the optimized 2-respecting cuts take $O(n(m\ +\ n))$ per tree. Consequently, the overall time complexity evaluates to $O(k\ \cdot\ (m \log m\ +\ n(m\ +\ n)))$, which simplifies to $O(k\ \cdot\ n(m\ +\ n))$.
- **Simplifications made:**
  Instead of implementing Gabow's exact tree-packing algorithm or a provable multiplicative-weights packing framework, the code relies on a straightforward greedy Kruskal load-based packing. It also replaces complex dynamic tree data structures like link-cut trees with standard static depth-first search traversals, preorder interval bounds (tin/tout), and linear bottom-up array aggregations. Furthermore, rather than maintaining strict determinism, it introduces controlled randomness by combining std::shuffle with std::stable_sort to randomize MST selections across equal-load weights.

- **What makes it different:** Unlike Karger's original contraction algorithm, which relies on random edge contractions to find minimum cuts, this approach utilizes a randomized greedy load-balancing heuristic across packed trees. Additionally, instead of requiring complex dynamic tree maintenance for cut evaluation, it computes 2-respecting cuts via a structural identity ($cut[a] + cut[b] - 2 \cdot cross[b]$) that is evaluated through a single efficient pass of static tree dynamic programming.

---

2. Theoretical Comparison and Predictions

Our project will be analysing the behaviour and performance of three distinct algorithms: Stoer-Wagner (deterministic), Karger (randomized) and Karger's tree packing (randomized). We aim to determine under which conditions each one of them outperforms the others, and if there is a clear criterion that helps us choose correctly between a randomized or deterministic approach in each circumstance.

First we will do a theoretical comparison of the algorithms.

2.1 Complexity, constants and memory

	Stoer–Wagner (matrix)	**Stoer–Wagner (priority queue)**	**Karger**	**Tree packing**
Guarantee	exact, deterministic	exact, deterministic	Monte Carlo; can only overestimate	Randomized Heuristic / Monte Carlo (High probability of success given a sufficient number of trees k)
Time of one run	$\Theta(n^3)$	$O(nm \log n)$	$O(m \log n)$	$O(k \cdot (m \log m +$ where k is num_trees
Probability one run is correct	1	1	$\geq \frac{2}{n(n-1)}$	$\geq 1 - 1/n^c$
Runs for failure $\leq \delta$	1	1	$\frac{n(n-1)}{2} \ln \frac{1}{\delta}$	$O(1)$ independent repeats

Total, sparse graph ($m = \Theta(n)$)	$\Theta(n^3)$	$O(n^2 \log n)$	$O(n^3 \log n)$	$O(k \cdot n^2)$
Total, dense graph ($m = \Theta(n^2)$)	$\Theta(n^3)$	$O(n^3 \log n)$	$O(n^4 \log n)$	$O(k \cdot n^3)$
Memory	$\Theta(n^2)$: an $n \times n$ table of *int*	$\Theta(n + m)$: hash maps, plus a heap rebuilt each phase	$\Theta(n + m)$: a copy of the edge list, plus union–find	$O(n + m)$
Main constant-factor costs	about n^3 very simple loop steps over plain arrays	hash-map lookups, heap pushes with stale entries	cheap runs, but many: $14,850$ at $n = 100$	Edge scans during 1-respecting cut size evaluation ($O(n \cdot m)$ per tree) and the edge-by-edge in_a checking loop to populate the opposite array ($n - 1$ times per tree).
Sensitive to	n only	n and density	time: n, m. Success rate: graph structure	k, n, m and random generation seed
Limits of our implementation	memory: about 400 MB at $n = 10,000$	input must have no self-loops	input must be connected	Greedy MST Heuristic: Uses randomized Kruskal load balancing instead of Gabow's exact matroid tree packing or multiplicative weights.

The Karger totals use the lecture's $\delta = 0.01$, i.e. $t = \frac{3n(n-1)}{2}$ runs.

Note on the two versions of Stoer–Wagner. Our deterministic algorithm is implemented in two versions. Both perform exactly the same steps: in each phase they build a maximum adjacency ordering, record the cut of the phase and merge the last two vertices. They differ only in how they find the vertex most strongly connected to those already added. The matrix version stores the graph as an $n \times n$ table of edge counts and scans every vertex at each step, which costs $\Theta(n^3)$ time and $\Theta(n^2)$ memory regardless of the number of edges. The priority-queue version stores only the existing edges in hash maps and uses a binary heap with lazy deletion, which costs $O(nm \log n)$ time and $\Theta(n + m)$ memory. The original algorithm reaches $O(nm + n^2 \log n)$ with Fibonacci heaps (Stoer and Wagner, 1997), but the binary heap is much simpler to implement, at the cost of an extra logarithmic factor. Since everything else is identical, comparing the two versions isolates the effect of the data structure.

2.2 Predictions

1. **Which algorithm wins on small dense graphs, and roughly where?**- On small dense graphs we expect the matrix version of Stoer–Wagner to be the fastest. Here, the asymptotic advantage of tree packing, $O(n^2 \log^3 n)$ against $\Theta(n^3)$, is outweighed by its much larger constant factors. Even ignoring constants, the two only meet when $n \approx \log^3 n$, which (taking logarithms base 2) happens around $n \approx 1000$. With tree packing's larger constants, the crossover should lie beyond the dense graph sizes we can store: a graph with n = 5000 and half of all possible edges already has over 6 million edges. We therefore place tree packing behind both versions of Stoer–Wagner at these sizes. The priority-queue version of Stoer–Wagner should be slower than the matrix version, since on dense graphs it does the same work plus hashing and heap operations, $O(n^3 \log n)$. Lastly we would have Karger, whose total time of $O(n^4 \log n)$makes it the slowest by far.

2. **Which wins on large sparse graphs?** - Here, the theory indicates large differences between the algorithms. The priority-queue version of Stoer–Wagner should clearly surpass the matrix version, since an n × n matrix is wasteful when m is small: $O(n^2 \log n)$ against $\Theta(n^3)$, a gap that grows with n. The matrix version will also reach a memory limit, since storing n² integers needs about 400 MB at$n = 10,000$. Tree packing, with $O(n \log^3 n)$, should become the fastest for large enough n (as long as our implementation keeps its near-linear behaviour). Karger, with $O(n^3 \log n)$, will be the slowest, and its large number of runs will restrict it to the smallest sizes we test.
3. **How will Karger's observed success rate compare with the 2/(n(n−1)) bound on cycles?** - Cycles are Karger's worst case in one specific sense: every vertex has degree exactly k = 2, so the graph has exactly $kn/2$ edges at every step, and each specific minimum cut survives with probability exactly $2/(n(n-1))$. However, a cycle has $n(n-1)/2$ minimum cuts, one for each pair of edges, and contracting an edge of a cycle always produces a smaller cycle. A run therefore always ends with two arcs joined by two edges, which is a minimum cut. We thus predict that Karger will find the minimum cut in 100% of runs on cycles, even with a single run.

4. **And on two-cluster graphs?** - We expect the success rate to be far above the bound. The k cut edges are a very small share of the $m = s(s-1) + k$ edges, and every vertex has degree at least s − 1, much larger than k, so the worst-case assumption is very pessimistic. As long as the cut survives, every super vertex lies inside one cluster, and any super vertex that is only part of a

cluster still has at least s − 1 edges to the rest of it. Writing $c = 2k/(s - 1)$, this gives $P(success) \geq \prod j/(j + c)$, $for\ j = 1, \ldots, n - 2$, which equals the general bound when K = s - 1. For s = 10 and k = 2, it predicts that at least 24% of single runs succeed, against 0.53% for the general bound. We therefore predict that the success rate decreases as k grows towards $s - 1$ and, for a fixed k, increases with the size of the clusters.

5. **How many repetitions will Karger need for 99% success, versus the theoretical formula?** - To achieve a success rate of at least 99%, that is, a failure probability δ = 0.01, we need

$$t \geq ln(1/\delta)/p = (n(n - 1)/2) \cdot ln\ 100 \approx 2.3\ n(n - 1)$$

runs, since t independent runs all fail with probability at most$(1 - p)^t \leq e\text{^}(- pt) \leq \delta$. For n = 20, this means 875 runs. However, this number comes from the worst-case p, so we predict that far fewer repetitions will be needed in practice: a single run on cycles, around 20 on two-cluster graphs with s = 10 and k = 2 (using the estimate from prediction 4), and far fewer than 875 on random graphs.

---

3. Experimental Methodology

3.1 Input families

Family	Generator	Parameters	Known min cut	Why it is included
Cycle	make_cycle(n)	n	2	A cycle has n(n−1)/2 minimum cuts. Each specific one survives a Karger run with probability exactly 2/(n(n−1)), but every run ends on one of them, so Karger never fails here
Complete graph	make_complete(n)	n	n−1	Densest possible input
Two clusters	make_two_clusters(s, k)	s, k	k	Unique, well-separated minimum cut
Random connected	make_random_graph(n, m, seed)	n, m, seed	computed by Stoer-Wagner	Controls density
Two Circulants	make_two_circulants(s, k)	s = n/2 and k =3	k	Two rings in which every vertex joins its two neighbours on each side (degree 4), connected by 3 edges. The minimum cut is unique and only slightly smaller than the vertex degrees, which is hard

Family	Generator	Parameters	Known min cut	Why it is included
				for random contraction to preserve

- *Success-rate experiment: n = 10, 20, 40, 80, all families.*
- *Time to 99% success: sparse (m = 3n) n = 20, 40, 80, 160; dense (m = n(n−1)/4) n = 20, 40, 80.*
- *Stoer-Wagner benchmark: sparse n = 50 to 1600, dense n = 50 to 400, density sweep at n = 300 from 1% to 100%.*
- *k-sweep: two clusters of 20 vertices, k = 1, 2, 4, 8, 12, 16, 18.*
- *Seeds: 2026 plus n (or m), as in the benchmark sources.*

3.2 Benchmark setup

- **Hardware:** Lenovo Legion Slim 5 16AHP9 laptop, AMD Ryzen 7 8845HS (8 cores, 16 threads), 16 GB RAM, Windows 11 Home (build 10.0.26200). All results come from this one machine, plugged in, in Quiet Mode, with other applications closed. Every algorithm runs on a single thread, so core count and the integrated and discrete graphics do not affect the results.
- **Compiler and flags:** MSVC 19.51.36257 for x64 (Visual Studio 2026), /std:c++17 /O2 /EHsc.
- **Timing method:** `std::chrono::steady_clock`. Graphs are built before timing starts, and only the call to the algorithm is timed.
- **Warm-up and repetitions:** one untimed warm-up run, then 5 timed runs for Stoer-Wagner and 3 for Karger and tree packing, which are much slower.
- **Variance handling:** the median is reported, and the minimum is also saved in the CSV. No confidence intervals on timings.
- **Randomized algorithms:** success rate is measured with 20,000 single runs (t = 1) per instance, each with a different seed, with 95% Wilson confidence intervals.
- In the timing experiment Karger is run $t = \lceil \ln(100) \cdot n(n-1)/2 \rceil$ times, the worst-case count for 99% success.

3.3 Fairness measures

- Same instances and seeds for every algorithm
- Stoer-Wagner serves as the exact answer for checking the randomized algorithms
- Stoer-Wagner is implemented with both a matrix and a priority queue version, so its comparison with algorithms that suit sparse graphs is not handicapped by a naive data structure. All three algorithms are C++17 implementations under the same interface, checked against the same exact answers.
- Graph generation and writing the CSV files happen outside the timed region. Karger's function copies the edge list once per call, which is included in its time but is negligible next to its thousands of runs.

3.4 Correctness testing

Stoer-Wagner - Both versions were checked against known answers (cycles, complete graphs, paths and stars for n = 3 to 60, and two clusters for every valid k), against a brute-force search over all bipartitions

on 520 random graphs with up to 14 vertices, and against each other on larger graphs up to n = 150. All checks passed.

Karger - tests/test_karger.cpp first checks properties that every single run (t = 1) must satisfy, with 20 seeds each: on cycles (n = 3 to 30) every run returns 2, because every run ends on one of the $n(n-1)/2$ minimum cuts; on random spanning trees (n = 2 to 30) every run returns 1; and on cycles with every edge doubled every run returns 4, which checks that the implementation handles multigraphs. On 46 random graphs, 2,300 single runs never returned less than the minimum cut from Stoer-Wagner, since every answer is a real cut. With $t = 3n(n-1)/2$ runs, Karger returned the known answer on complete graphs (n = 4 to 20) and on two clusters (s = 5 to 12, every valid k), and matched Stoer-Wagner on three random graphs. All tests passed, and in the timing experiment it returned the exact minimum cut on all seven instances.

3.5 Reproducing the results

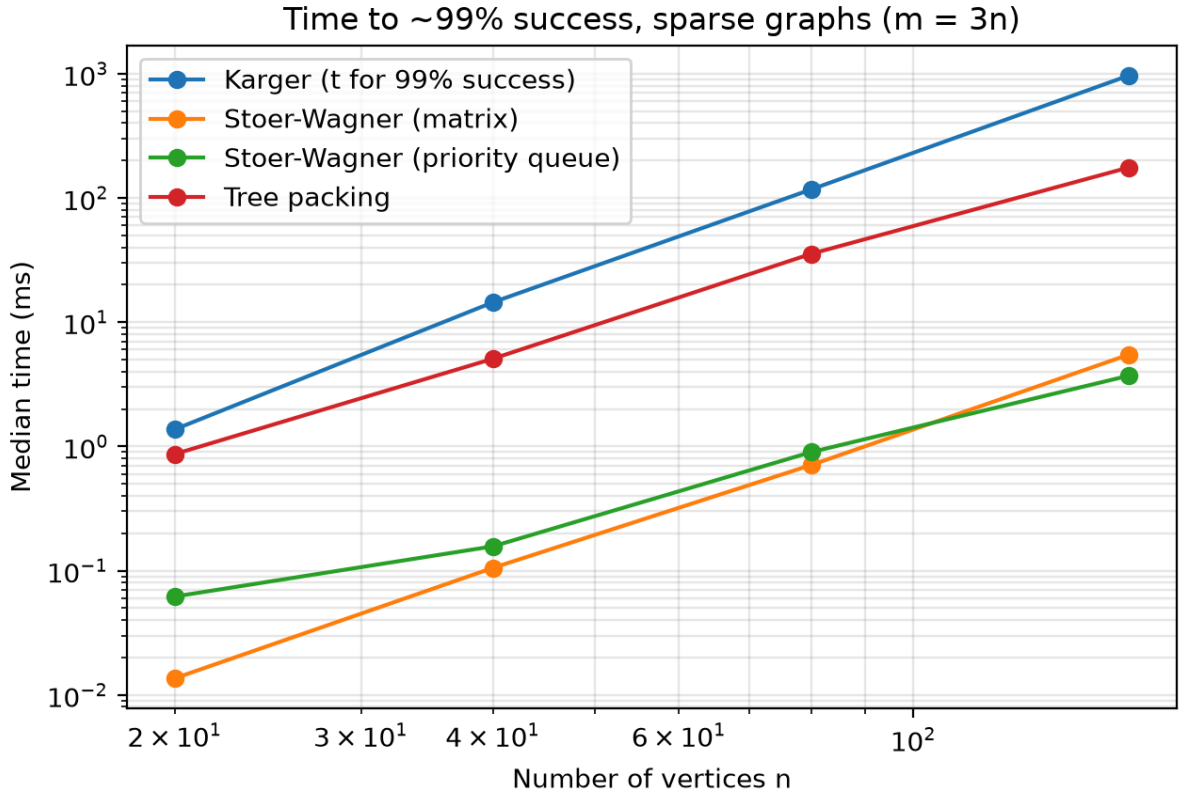
Exact commands to build and run can be found in the README, and the CSVs and plots are written in the results folder after execution of the code.

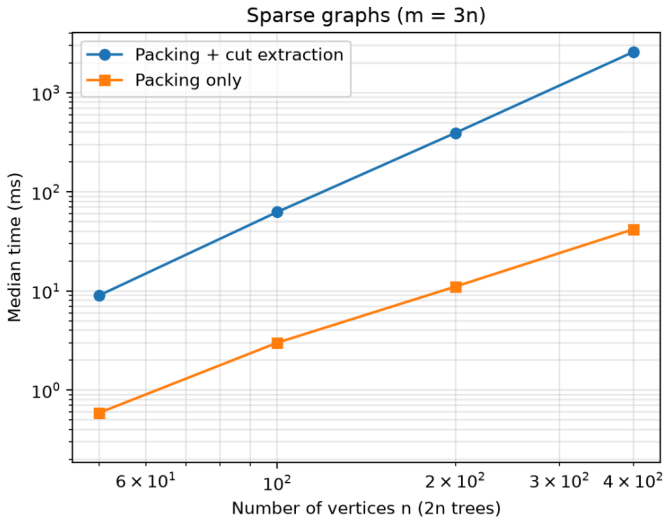
---

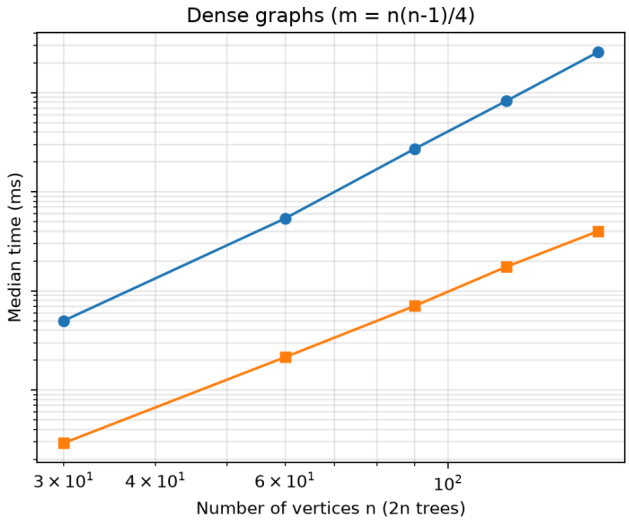
4. Results and Analysis

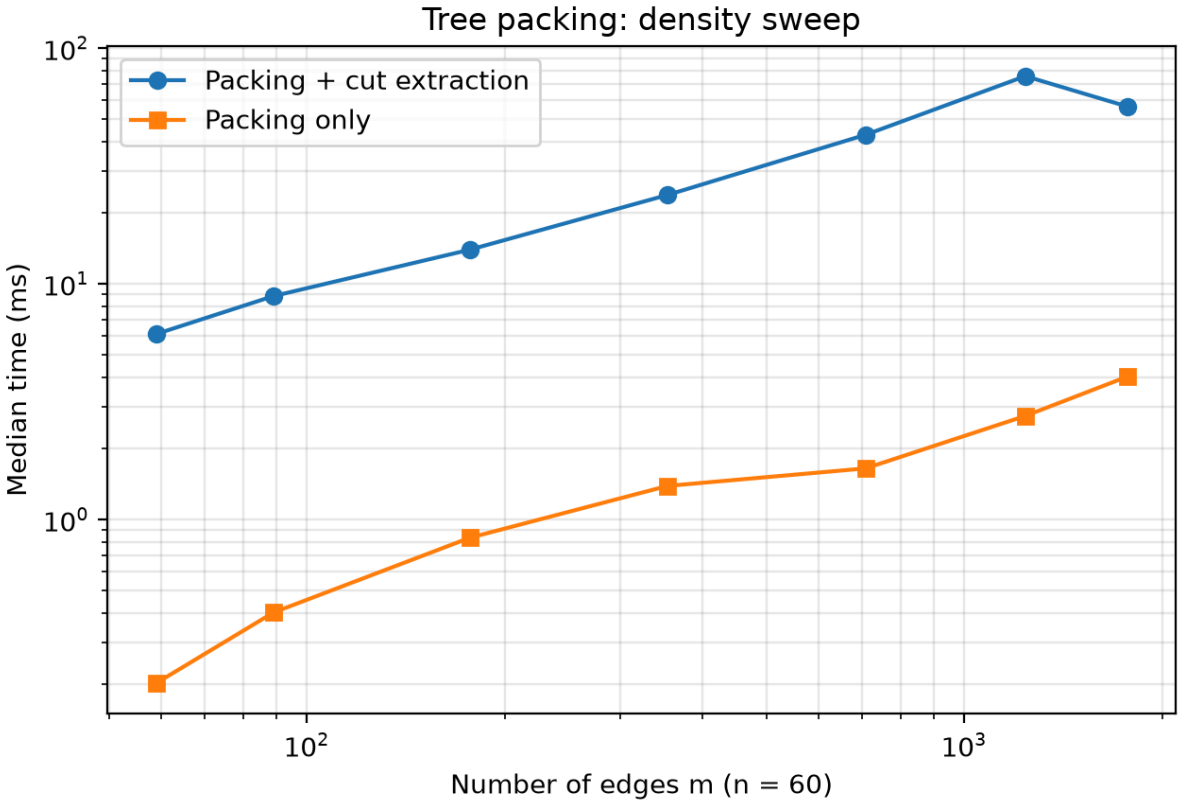
All results were produced on one machine (Section 3.2). Every algorithm returned the exact minimum cut on every instance of the timing experiment (28 of 28 runs), and tree packing was exact on all 26 instances of its own benchmark.

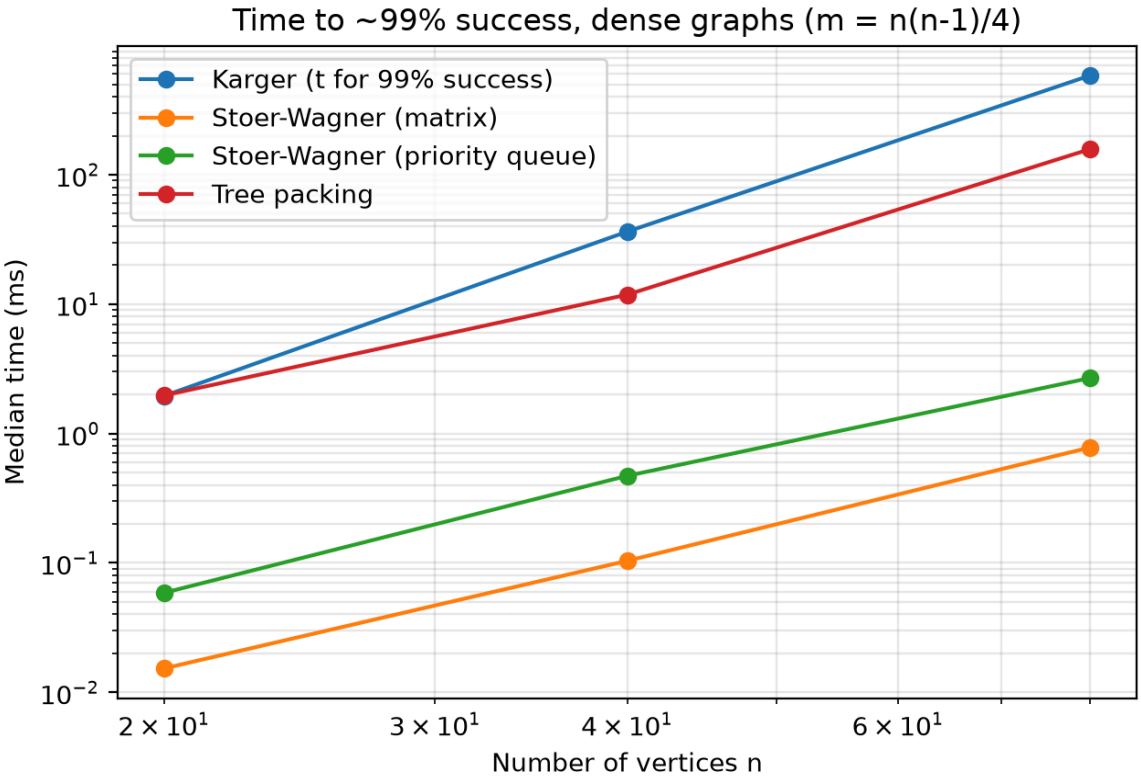
4.1 Running time versus graph size and density


Time to ~99% success, sparse graphs (m = 3n)
Karger (t for 99% success)
Stoer-Wagner (matrix)
Stoer-Wagner (priority queue)
Tree packing
Median time (ms)
Number of vertices n


Sparse graphs (m = 3n)
Packing + cut extraction
Packing only
Median time (ms)
Number of vertices n (2n trees)


Dense graphs (m = n(n-1)/4)
Median time (ms)
Number of vertices n (2n trees)


Tree packing: density sweep
Packing + cut extraction
Packing only
Median time (ms)
10^2
10^1
10^0
10^2
10^3
Number of edges m (n = 60)


Time to ~99% success, dense graphs (m = n(n-1)/4)
Karger (t for 99% success)
Stoer-Wagner (matrix)
Stoer-Wagner (priority queue)
Tree packing
Median time (ms)
10^2
10^1
10^0
10^{-1}
10^{-2}
2×10^1
3×10^1
4×10^1
6×10^1
Number of vertices n

Time to about 99% success (ms, median; Karger runs the worst-case number of repetitions, tree packing uses 2n trees):

Graph	n	Stoer-Wagner matrix	Stoer-Wagner PQ	Tree packing	Karger
Sparse (m = 3n)	20	0.014	0.062	0.87	1.4
	40	0.105	0.157	5.1	14.4
	80	0.71	0.90	35.4	116.8
	160	5.5	3.7	175.9	966.9
Dense (m = n(n−1)/4)	20	0.015	0.059	2.0	2.0
	40	0.104	0.47	11.8	36.3
	80	0.78	2.7	157.8	585.1

Fitted growth exponents (time $\propto n^e$) over these sizes:

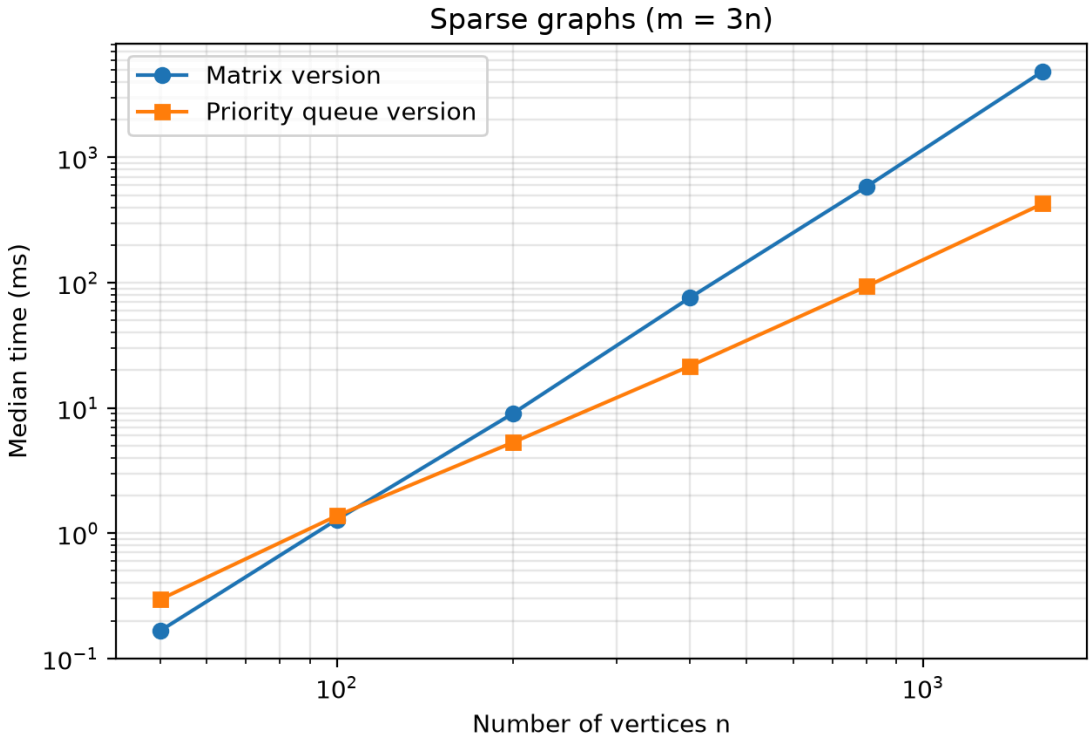
	Matrix	PQ	Tree packing	Karger
Sparse	2.87	2.02	2.58	3.14
Dense	2.84	2.76	3.16	4.11

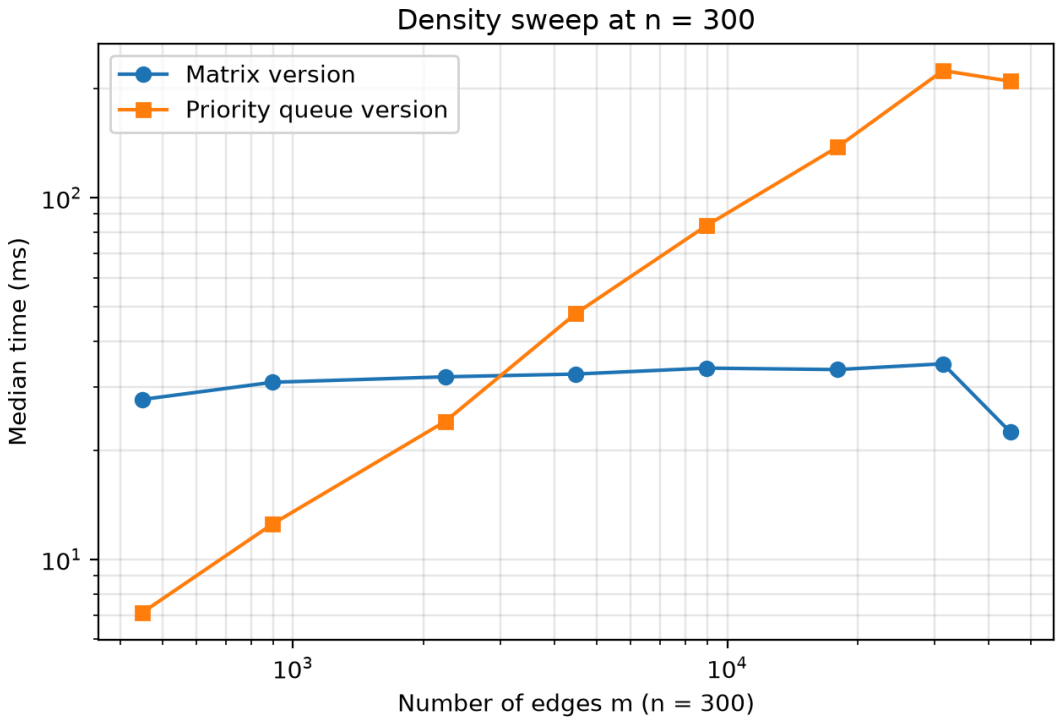
The exponents follow the structure of each algorithm. The matrix version always scans n vertices per selection, n selections per phase and n−1 phases, so it is $\Theta(n^3)$ whatever the density. Karger needs about n^2 runs, each costing O(m log n), giving n^3 log n on sparse graphs and n^4 log n on dense ones, which matches 3.14 and 4.11. The priority queue version costs O(nm log n) and so grows like n^2 on sparse graphs, which matches 2.02. Tree packing with 2n trees costs $O(n \cdot n(n+m))$, so about n^3 on sparse and n^4 on dense graphs. Its measured exponents (2.58 and 3.16) are lower, probably because at $n \leq 160$ the lower-order terms (sorting edges, building trees) still take a noticeable share. With n only up to 160 for the 99%-success comparison, these exponents have not reached their asymptotic values.

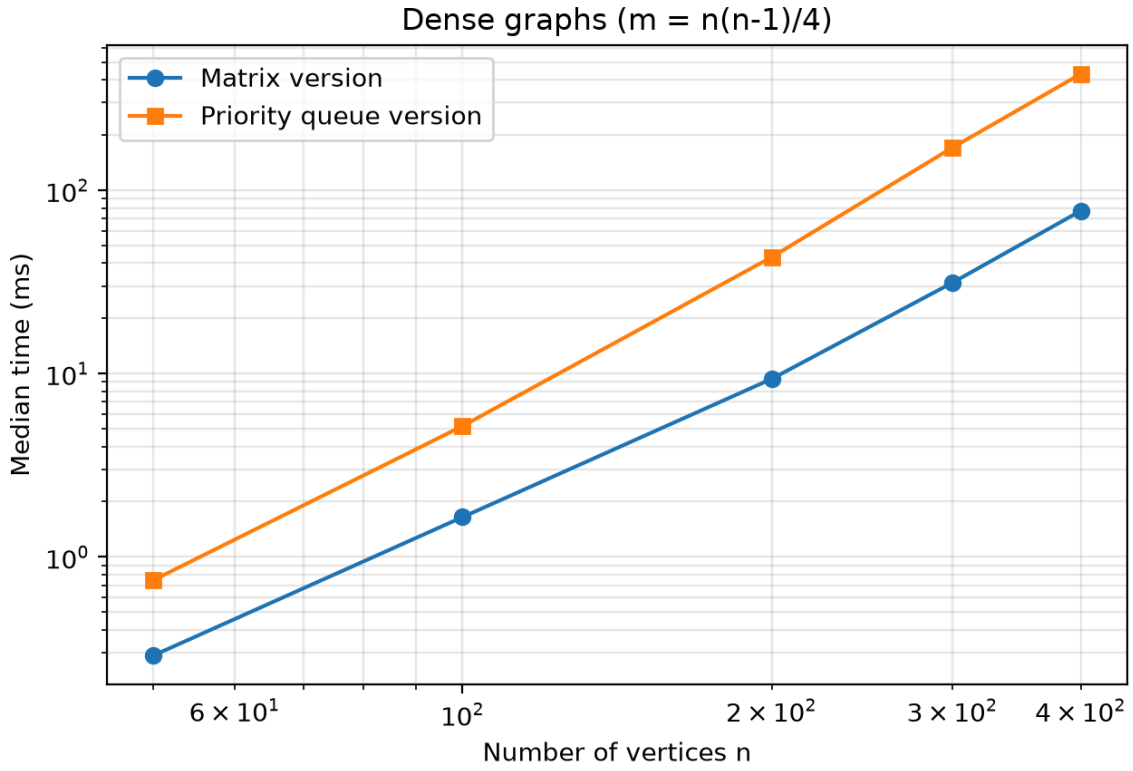
In tree packing's own benchmark, 89% to 98% of the time goes on extracting the cut, not on building the packing, and the share grows with n (98.4% at n = 400). Extraction costs O(nm) per tree for the 1-respecting cuts plus O(n(n+m)) for the 2-respecting ones, while packing is a sorted Kruskal pass. Time

is linear in the number of trees (0.18–0.20 ms per tree at n = 80 from 40 to 320 trees). With n fixed at 60, time grows about 12× while m grows 21× (m = 59 to 1239), as expected once m dominates n.

4.2 Stoer-Wagner: matrix versus priority queue







The two versions run the same phases and differ only in how they find the next vertex.

Sparse (m = 3n)	n=50	100	200	400	800	1600
Matrix (ms)	0.17	1.28	9.1	75.8	583	4866
PQ (ms)	0.30	1.38	5.3	21.6	93	427
PQ ÷ matrix	1.78	1.07	0.59	0.28	0.16	0.088

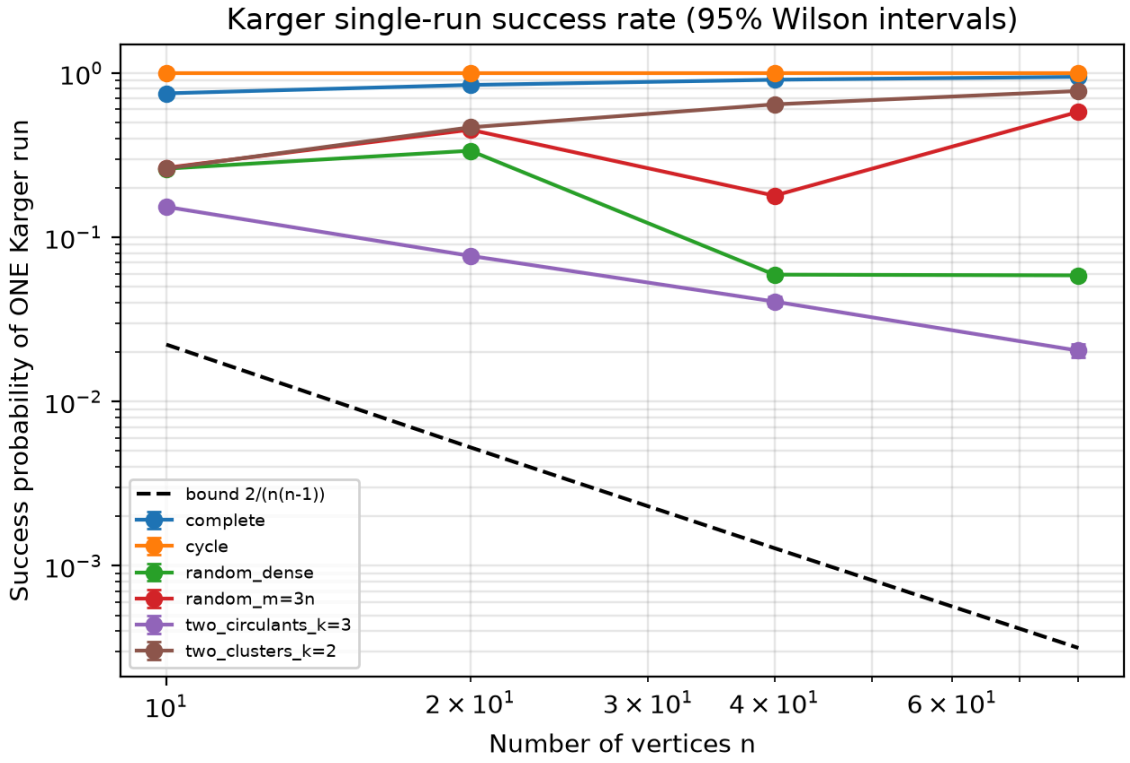
Density sweep, n = 300	m=449	897	2243	4485	8970	17940	31395	44850
Matrix (ms)	27.6	30.8	31.9	32.5	33.8	33.4	34.7	22.5
PQ (ms)	7.1	12.5	24.1	47.8	83.7	137.9	224.5	209.7
PQ ÷ matrix	0.26	0.41	0.75	1.47	2.48	4.12	6.47	9.33

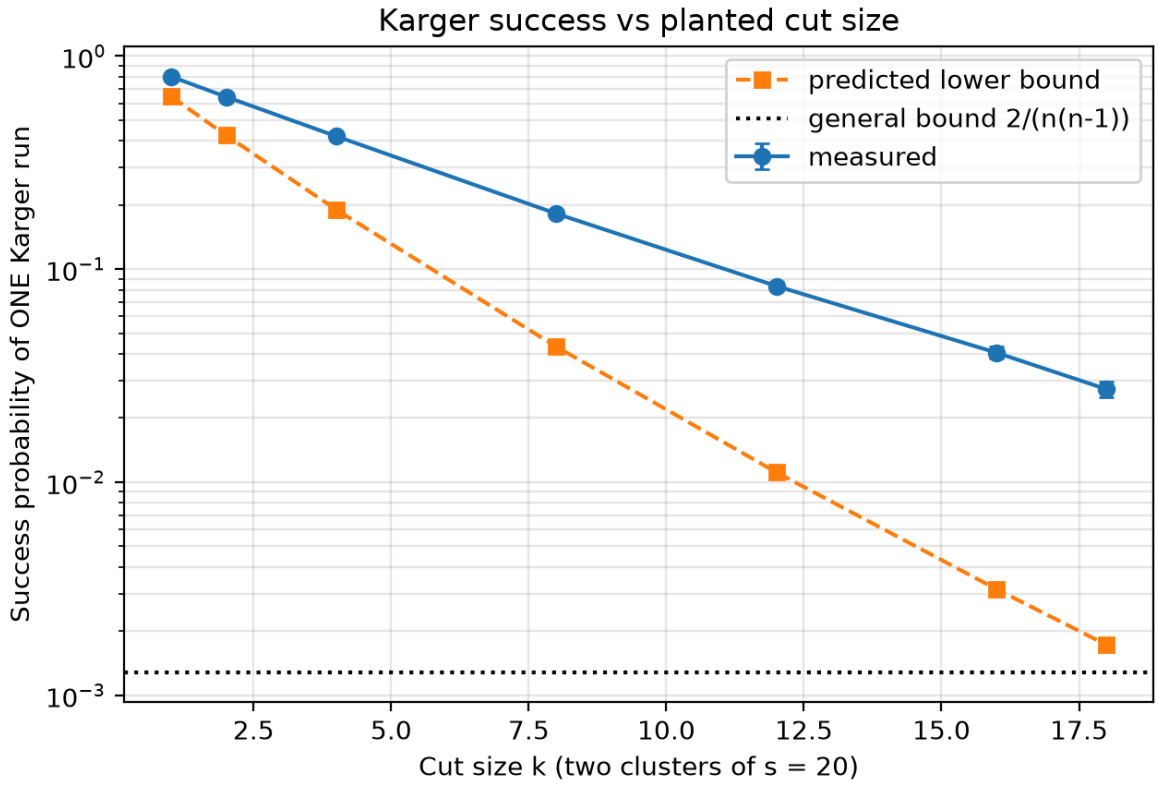
On sparse graphs the exponents are 2.96 (matrix) and 2.08 (priority queue), against the predicted 3 and 2. The matrix version scans n entries per selection however few edges exist, so its time does not depend on m, which is why the density sweep is flat at about 28 to 35 ms. The priority queue version does work proportional to the edges, so it grows steadily with m. It also has a floor: each phase pushes every remaining vertex onto the heap, so it does $\Omega(n^2 \log n)$ work even on a sparse graph.

The crossover has two causes. At small n or high density the priority queue loses because each of its steps (hashing an edge, pushing onto a heap, discarding stale entries) costs more than one step of the matrix scan over a contiguous array. At large n and low density it wins because it skips the n^2 scan. In our measurements the crossover on sparse graphs (m = 3n) is near $n \approx 100$. At n = 300 it is near $m \approx 3000$ edges, which is a density of about 6.7%, or an average degree of about 20. On dense graphs the matrix version always wins, by 2.6× at n = 50 growing to 5.6× at n = 400, matching the predicted extra log factor and heap overhead.

One result we cannot explain: at the complete graph the matrix version got faster (34.7 to 22.5 ms) although it performs the same operations as for any other density. Tree packing shows a similar dip at its complete graph (75.8 to 56.4 ms at n = 60). A possible cause is that make_complete lists edges in a regular order while the random generator does not, which could change memory access or branch prediction. We did not test this.

4.3 Karger's success probability in practice versus the theoretical bound





Success rate of a single run (20,000 runs per point; 95% intervals are narrower than ±0.007):

Family	n=10	20	40	80	Fitted slope
Bound 2/(n(n−1))	0.0222	0.0053	0.0013	0.0003	−2
Cycle	1.000	1.000	1.000	1.000	
Complete	0.751	0.845	0.909	0.947	+0.11
Two clusters (k = 2)	0.262	0.466	0.644	0.776	+0.52
Two circulants (k = 3)	0.153	0.077	0.041	0.020	−0.96
Random (m = 3n)	0.265	0.451	0.179	0.580	
Random (dense)	0.261	0.336	0.059	0.059	−0.90

No family came near the bound. On cycles, each specific minimum cut survives with probability exactly 2/(n(n−1)), but a cycle has n(n−1)/2 minimum cuts and contraction always ends on one of them, so every run succeeds. On complete graphs, all n single-vertex cuts are minimum cuts, and success rises with n. On two clusters with k fixed, the planted cut becomes a smaller share of the edges as clusters grow, so the success rate rises.

The two-circulants family was designed to be hard: a unique minimum cut of 3 while every vertex has degree 4. Its success rate decays like 1/n (slope −0.96), not $1/n^2$, and at n = 80 it is 65 times above the bound. Our explanation, from a back-of-envelope estimate that has not been verified: once contraction merges vertices into arcs of a ring, each arc has 6 edges leaving it. That makes each step's chance of hitting the 3 cut edges about 3/(3r), so the survival probability is the product of (1 − 1/r) over r = 3 to n, which equals 2/n. That gives 0.100, 0.050 and 0.025 for n = 20, 40 and 80, against measured 0.077, 0.041 and 0.020. The random families are not monotone because each size is a single random graph with its own minimum cut (3, 3, 2, 2 for the sparse family), so trends there mix size and instance effects.

On two clusters of 20 vertices (n = 40) with growing k, success falls roughly exponentially:

k	1	2	4	8	12	16	18
Measured	0.801	0.644	0.420	0.182	0.083	0.0405	0.0273

Section 2.2 formula	0.646	0.424	0.190	0.043	0.011	0.0031	0.0017
Direct product $\prod(1 - c/r)$	0.745	0.552	0.300	0.083	0.021	0.0045	0.0020

The formula held as a lower bound at every k, but it is loose, and the looseness grows with k (1.2× at k = 1, 16× at k = 18). The direct product is tighter at small k (1.08×) but is still 14× low at k = 18. For s = 10, k = 2 the report predicted at least 24%, and we measured 46.6%.

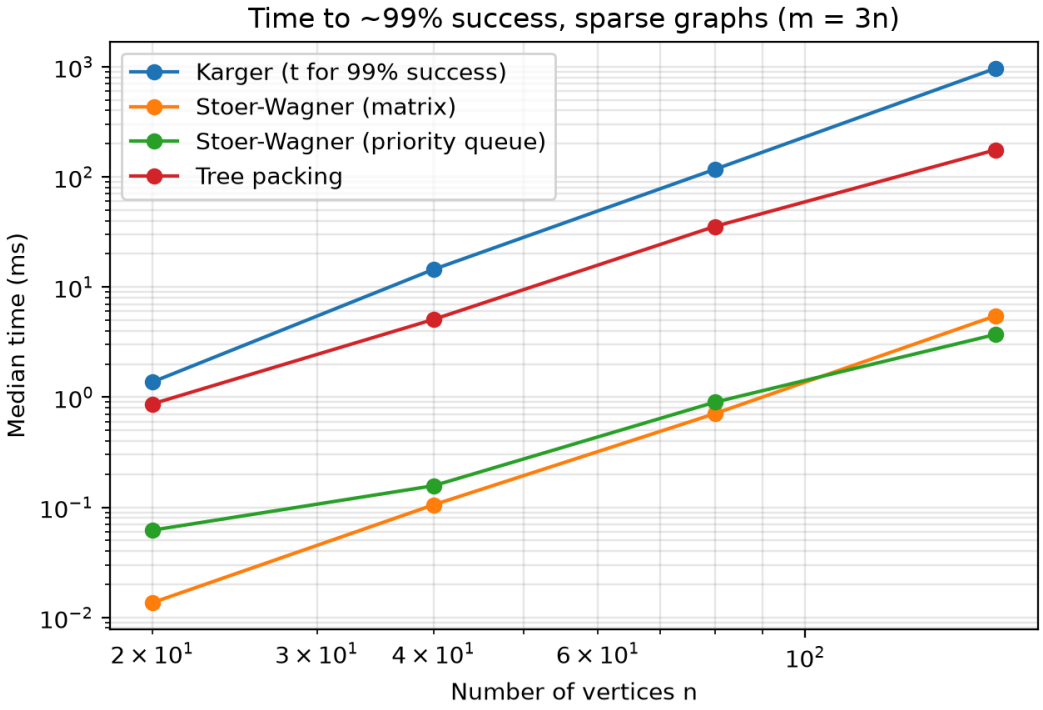
4.4 Repetitions needed for a target success probability

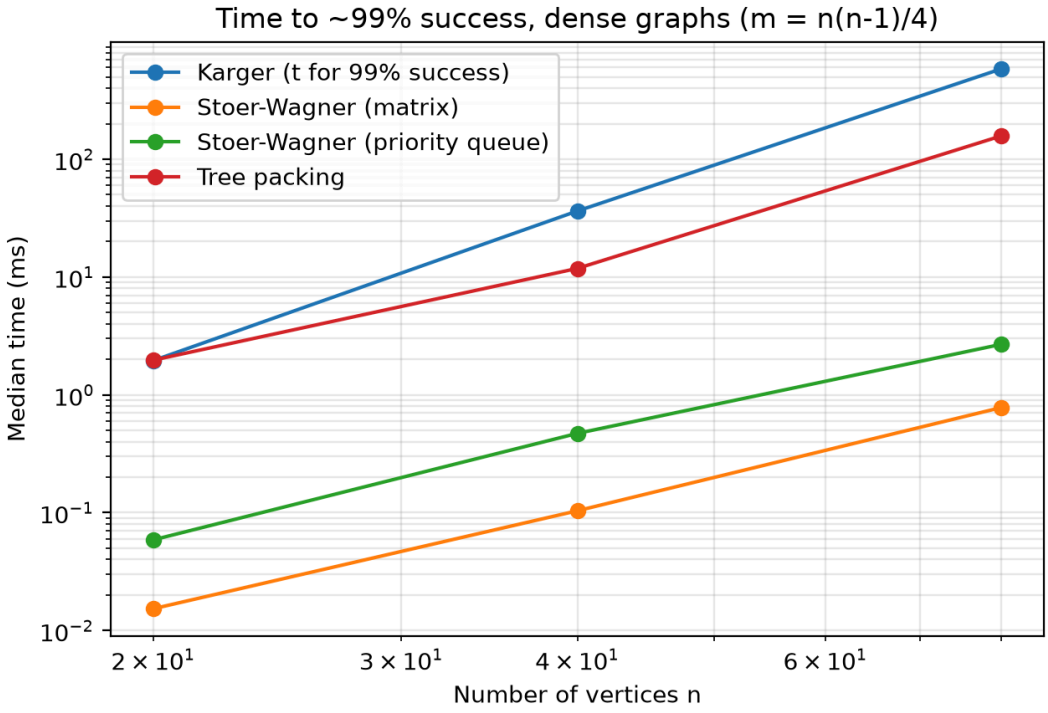
Runs needed for 99% success, computed from the measured single-run rate as $\ln(0.01)/\ln(1 - \hat{p})$ (round up for a real count):

	n=10	20	40	80
Formula $\ln(100)\cdot n(n-1)/2$	207	875	3592	14,552
Cycle	1	1	1	1
Complete	3.3	2.5	1.9	1.6
Two clusters (k = 2)	15.1	7.3	4.5	3.1
Random (m = 3n)	15.0	7.7	23.3	5.3
Random (dense)	15.2	11.2	75.5	76.3
Two circulants (k = 3)	27.7	57.5	111	223

The formula is safe but pessimistic, by a factor of 7 at n = 10 and 65 at n = 80 for the worst family. The real number depends on the graph's structure and not just on n: it falls with n for complete graphs and two clusters, and grows roughly linearly for the circulants. The formula grows with n^2, so the gap widens. For two clusters with s = 10 and k = 2, the report predicted about 20 runs and 7.3 were needed. The formula is the only value that is safe without knowing the graph.

4.5 Total time to reach a fixed success probability





Using the table in 4.1, on sparse graphs at n = 160 the order is priority queue (3.7 ms), matrix (5.5 ms), tree packing (176 ms), Karger (967 ms). Karger is 260× slower than the priority queue, and tree packing is 47× slower. On dense graphs at n = 80, Karger is 749× slower than the matrix version (585 vs 0.78 ms) and tree packing is 202× slower. The gaps grow with n, because Karger's exponent (3.14 sparse, 4.11

dense) exceeds Stoer-Wagner's. Stoer-Wagner's cost is fixed: one pass with no repetitions and no success probability to manage.

Two things about this comparison favour Stoer-Wagner more than a fair setting would:

- **Karger's repetitions are the worst-case formula.** Section 4.4 shows 7 to 65 times fewer runs are needed in practice, and the per-run cost is about 8 µs at n = 80 sparse. Needing only 5.3 runs (the random family) would cost about 0.04 ms, below Stoer-Wagner's 0.7 ms. Needing 223 runs (the circulants) would cost about 1.8 ms, above it. This is a projection from per-run cost, not a measurement, and it assumes the success rate is known in advance. The slowness comes from the guarantee, not from the algorithm.
- **Tree packing uses 2n trees, which is a chosen setting.** The tree-count sweep reached 99% with 1 to 4 trees on all five hard instances, and per-tree time is constant, so 2n trees is far more than needed. We did not rerun the timing with fewer trees, so the 47× gap is for the conservative setting.

Neither caveat changes the main result at these sizes: the deterministic algorithm is fastest at every size we measured. They do mean the margin over the randomized algorithms is smaller than the plots suggest.

4.6 Comparison against the predictions in Section 2

Prediction	Outcome	Notes
1. Small dense graphs: matrix fastest, then priority queue, then tree packing, Karger slowest	Confirmed	Dense n=80: matrix 0.78 ms, PQ 2.7 ms, tree packing 158 ms, Karger 585 ms
2a. Sparse: priority queue overtakes matrix	Confirmed	Crossover near n≈100. At n=1600 the priority queue is 11× faster (427 vs 4866 ms). Exponents 2.08 and 2.96, against the predicted 2 and 3.
2b. Sparse: tree packing becomes fastest for large n	Not observed	At n = 160 it is 47× slower than the priority queue, and the gap grows (14× at n=20, 32× at 40, 39× at 80, 47× at 160). This used 2n trees (Section 4.5).

Prediction	Outcome	Notes
2c. Matrix hits a memory limit at n=10,000	Not tested	The largest n run was 1600
3. Cycles: Karger succeeds 100%	Confirmed	1.0000 at every n, so 1 run is enough
4. Two clusters: far above the bound, falls with k, rises with cluster size	Confirmed	k=2: 0.26, 0.47, 0.64, 0.78 for n=10, 20, 40, 80. For s=10, k=2 the report predicted at least 24%, and the measurement is 46.6%.
5. Repetitions far below the formula	Confirmed	n=20: formula 875, observed 1 to 57; for two clusters (s=10, k=2) about 20 predicted, 7.3 observed. n=80: formula 14,552, worst observed 223 (Section 4.4).

---

5. What Surprised Us

5.1 Karger almost always finds the minimum cut of a complete graph

- **What we expected:** In a complete graph every vertex has degree $n - 1$, exactly the minimum cut, which is the condition under which each step of the bound in Section 1.3 is tight. We expected it to be one of Karger's hardest inputs, with a success rate near $2/(n(n-1))$, which is 0.03% at$n = 80$.
- **What happened:** A single run found the minimum cut 75.1% of the time at n = 10, 84.5% at 20, 90.9% at 40 and 94.7% at 80 (Section 4.3): success rose with n while the bound fell. At n = 80, 99% success needs fewer than 2 runs instead of 14,552.
- **Our explanation:** the worst case is only true in the first step. After that, two effects work in Karger's favour. First, every single vertex is a minimum cut, so the run has n minimum cuts to end on. Secondly, the number of edges between two super-vertices is the product of their sizes, so a super-vertex of size s has degree $s(n - s)$, far above $n - 1$ once $s \geq 2$.. As the algorithm advances, random edges are therefore more likely to touch the largest super-vertex, and the run will most likely end with one giant super-vertex and one lone vertex, which is a minimum cut. Even with a giant and two lone vertices left, the run only fails with probability$1/(2n - 3)$, when the edge between the two lone vertices is contracted.

- **How we checked the explanation:** we wrote a small check (bench/check_giant.cpp) that runs the same contraction as karger_alg but stops when a given number of super-vertices remain and records the size of the largest one, over 2,000 runs with n = 80. On the complete graph a giant forms early: with 10 super-vertices left, the largest already holds 68.9 of the 80 vertices on average, and with 3 left, the run is a giant plus two lone vertices in 87.5% of runs. On the two-circulants graph no giant forms: with 10 left the largest holds only 34 vertices, and the giant-plus-lone-vertices pattern appears in only 17.4% of runs with 3 left. This matches our explanation.

---

6. What We Would Do Next

- Karger–Stein (Karger and Stein, 1996), which repeats only the risky final contractions, to see whether its $O(n^2 \log^3 n)$ total time closes the gap with Stoer-Wagner.
- The full near-linear tree packing, with random sampling and link-cut trees, at larger n to test prediction 2b, and timed with fewer than 2n trees (Section 4.5).
- Several random graphs per size instead of one, which made the random families non-monotone (Section 4.3).
- The check from Section 5.1 on other dense graphs, such as two clusters, to see whether a giant super-vertex is the general reason dense graphs are easy for Karger.
- Weighted graphs, where Karger would pick edges with probability proportional to their weight.

---

7. Individual Contributions

Aarav Khemani: Primary author of Stoer-Wagner (both versions) and its test file. Also worked on sections of the report including making the primary file and structure of it. Created the benchmark and plotting files for the Stoer-Wagner and the comparative files, Also created the README and requirements files.

Rodrigo Uzal Sequeros: Primary author of Karger's contraction algorithm (karger/karger.hpp) and its test file (tests/test_karger.cpp). Also wrote the shared graph generators (graphs/graphs.hpp), added the k-sweep experiment to the comparative benchmark (Experiment 3 in bench_all.cpp, with its plot in plot_all.py) and wrote the giant super-vertex check (bench/check_giant.cpp). In the report, sections 1.1, 1.3, 2.2, 5 and 6 and the Karger parts of 2.1 and 3.4. For the video, prepared the shared script, the recording plan and the slide template, and recorded the Karger deep dive, the success-rate results and the most surprising finding.

Rentaro Beattie: Primary author of karger's tree packing algorithm (karger_tree_packing.hpp, bench_karger_tree_packing.cpp, and appropriate report sections). Created the slides, script and voiceovers for section 4 and 5a (tree packing and runtime). Created the github repository. Edited and submitted the final video.

---

8. AI Use

8.1 Per-member disclosure

Aarav Khemani: AI was used heavily in my implementation and parts of the assignment, including the report and formatting. Claude (Anthropic, Sonnet 5.5) was my choice of AI, which I used to understand the assignment and the three algorithms and to plan the project; to write both Stoer-Wagner implementations and my test file, which checks them against known answers, a brute-force search and each other; to write the benchmark programs (bench_stoer_wagner.cpp and bench_all.cpp), the Python plotting scripts, the README and requirements.txt; to find bugs in the shared graph code (a missing semicolon in make_two_clusters, and make_complete producing duplicate edges and self-loops) and to set up MSVC and VS Code. Also was used to draft the report template.

Rodrigo Uzal Sequeros: I used Claude (Anthropic, Opus 5.5) throughout the project as an assistance and support tool. In my code, it helped me correct and revise my Karger implementation. It also wrote the plotting code for the k sweep (marked in plot_all.py), and the check in bench/check_giant.cpp, which I retrieved, revised and ran. It also helped me reflect on the distinct test that I wanted Karger to be tested against and correct the implementation of these. For the report, the two-cluster prediction formula in Section 2.2 was derived with its help as well as revising my sections as I went along and wrote them. It also specifically helped me write the table for the complexity of the three algorithms which later my colleagues revised. The PowerPoint presentation that I used as visual support for my parts of the video was designed with the help of Claude, I described what i wanted to explain and i wanted to be shown on screen and it helped me create the slides, which saved a lot of useful time for redacting a better script instead of designing simple graphs with the tedious interface of the app which would have led to a great waste of time.

Rentaro Beattie: Deepseek v4.1 flash was used to implement the algorithm and directly compare it to the paper's implementation. Then I used Google gemini 3.1 initially to research the topic, schedule tasks and help with formatting, later on during the production of the slides, gemini was used to generate the diagrams for the slides and convert the report sections into a readable script. I finally used the same deepseek session (that was used to implement the algorithm) to help me identify the worst cases and bugs to write my individual reflection.

8.2 Where AI was wrong, unhelpful or misleading (at least three examples across the group)

#	Member	What the AI did wrong	How we noticed	What we did about it
1	Aarav Khemani	Claude suggested cycle graphs as the case where Karger's success rate approaches the 2/(n(n−1)) bound, and put that in the report's input-family table.	Rodrigo's derivation showed every run on a cycle ends on a minimum cut, and the benchmark measured 100% success at every size.	Corrected the table, and added a family with a unique minimum cut (two circulants) to probe the bound properly.

#	Member	What the AI did wrong	How we noticed	What we did about it
2	Rodrigo Uzal Sequeros	When building the slides, Claude wrongly decided that the bowtie bug in our video script could not have happened, because it assumed my counting loop scanned all the edges, so a real bug was left out of the video.	Writing my individual reflection, I checked my first version: its counting loop also stopped after a fixed number of steps, which is why it returned 0.	The bug is described in my reflection, and the video shows the corrected run.
3	Rodrigo Uzal Sequeros	In the group's video script, Claude wrote that Karger needed "3 to 165" runs for 99% success in the k sweep.	When the slides were rebuilt from bench_ksweep.csv, the largest value was 166.4 runs, which rounds up to 167.	We corrected it to 167 and took every number on the slides directly from the CSV files.
4	Rentaro Beattie	Deepseek generated a deterministic algorithm for tree packing instead of the required randomised algorithm.	When Aarav reviewed the code during benchmark creation	Went through the code and made suggestions to the agent on how it could convert the algorithm with the least change

8.3 AI in group work

- **Where it helped the group function:** we recorded our segments separately, but one shared script with timings and one slide template, both drafted with AI, kept the video consistent in style, length and terminology
- **Where it hurt or threatened:** we used different tools (Claude, DeepSeek and Gemini), so AI-written text in shared documents could carry claims nobody had checked, like the bowtie bug in 8.2, which almost reached the video. Terminology and notation also drifted between sections. We handled this by making each primary author responsible for checking every statement about their own algorithm against the code, the tests and the CSV files.

9. Individual reflections:

Individual Reflection: Karger's Algorithm
Rodrigo Uzal Sequeros

1. **Worst case and the input that triggers it**

As we have seen in class, Karger's algorithm succeeds only if it never contracts an edge of the minimum cut, so the smaller the number of cut edges, the greater the chance of success.

Let G be a simple graph with n vertices and a minimum cut of k edges. Every vertex has $deg(v) \geq k$, since otherwise its star cut would be better than the minimum cut, so G has at least $nk/2$ edges. As contraction cannot decrease the minimum cut, this still holds after each step, so at step i, with $n - i + 1$ vertices left, a cut edge is chosen with probability at most $k / (k(n - i + 1)/2) = 2/(n - i + 1)$. Multiplying the n − 2 steps, the product telescopes:

$$Pr[a\ fixed\ minimum\ cut\ survives] \geq \prod (from\ i = 1\ to\ n - 2)\ of\ (n - i - 1)/(n - i + 1)$$

$$= (n - 2)/n \cdot (n - 3)/(n - 1) \cdots 2/4 \cdot 1/3 = 2/(n(n - 1)).$$

This bound is reached exactly when every vertex has degree exactly k, so the graph has only $nk/2$ edges at every step, as in the cycle graph, where each specific minimum cut survives with probability exactly $2/(n(n - 1))$. In most graphs the cut edges are a smaller share, so the minimum cut is more likely to survive.

2. Hardest bug and how I resolved it

My implementation has one structure (UnionFind) and two functions: karger_alg, which performs one run, and karger_min_cut, which repeats it t times and keeps the smallest cut. Following the class slides, rather than choosing a random edge at each step, I randomly order the edges beforehand and contract them in that order.

The hardest bug came from this decision. My first contraction loop ran for a fixed number of steps (i < n − 2), assuming every edge produces a contraction. However, when both endpoints already belong to the same super vertex, the edge is a self-loop and is skipped. On the bowtie graph (two triangles sharing vertex 2) with the order (0, 1), (1, 2), (0, 2), (3, 4), (2, 3), (2, 4), the loop stopped with three groups instead of two, and since the counting loop had the same problem, the function returned a 0 cut instead of 2. I solved it by stopping only when uf.components reaches 2 and counting crossing edges over all m edges with find. As I was unfamiliar with randomness in C++, I used Claude Opus 5.5 for the random number generator and the shuffle.

3. What I would do differently

First of all, I would have continuously tested the code with small tests, since a small graph with a known answer, like the bowtie, would have exposed the bug above straight away. Secondly, I would agree on the shared interface with my teammates before starting, as I had to reorder the parameters of karger_min_cut afterwards to match the group's (n, edges) format. Finally, I would add path compression to find, which flattens the union-find trees and makes each search almost constant instead of O(log n), and measure how much it actually helps.

4. What I now understand

At first I did not fully understand why $t = 3n(n - 1)/2$ repetitions bound the failure probability by e^{-3}. Writing $p = 2/(n(n - 1))$, so that $t = 3/p$, and using $1 - p \leq e^{-p}$:

$$Pr[all\ t\ runs\ fail] \leq (1 - p)^t \leq e^{-pt} = e^{-3} \approx 0.05.$$

I also realised that $3/p$ is not the smallest valid choice: failure below 1/10 only needs $pt \geq ln\ 10 \approx 2.3$, so 3 is just a convenient rounded-up value.

Individual Reflection: Karger's Tree-Packing Algorithm

Rentaro Beattie

1. Worst Case and Input Triggers

From a performance perspective, the worst-case input for our implementation is a dense graph where m = Θ(n²). Because the 2-respecting cut evaluation loops through every graph edge to build the opposite array for each subtree a, the work per tree is O(n · (m + n)), which simplifies to O(n³). Across k packed trees, the total time complexity scales to O(k · n³). Structurally, the algorithm struggles most on highly connected graphs with multiple near-minimum cuts, such as dense clustered networks or circulant graphs. On these, greedy Kruskal load-balancing tends to disperse edge usage across the tight bridge edges during early tree construction. If the number of trees k is too small, edge loads on the true min-cut rise rapidly, causing later trees to completely miss the min-cut or cross it in 3 or more edges thereby breaking the 2-respecting invariant.

2. Hardest Bug / Design Decision & Resolution

The single hardest design hurdle was figuring out how to evaluate 2-respecting cuts efficiently without implementing heavy dynamic tree data structures like Link-Cut Trees or Top Trees. A naive brute-force check of every edge pair and cut recalculation from scratch is prohibitively slow at O(k · n²m). The breakthrough came from deriving an algebraic inclusion-exclusion identity for the middle cut piece formed by subtrees A and B:cut(middle) = cut(A) + cut(B) − 2 · cross[b] Implementing this correctly required handling ancestry edge cases cleanly—specifically, skipping pairs where B is an ancestor of A (if (is_ancestor(b, a)) continue;) since those are naturally covered when A and B swap roles. By precomputing opposite[v] counts and aggregating them bottom-up in reverse preorder (cross[parent[v]] += cross[v]), we reduced 2-respecting cut evaluation to O(1) per edge pair.

3. What I Would Do Differently From Scratch

If I were to rewrite this implementation from scratch, I would optimize the O(n · m) bottleneck present in the opposite array construction. Right now, for every child node a, we blindly scan all m graph edges to test if (in_a[e.first] != in_a[e.second]). Instead, I would maintain an explicit adjacency list of non-tree graph edges. By leveraging our preorder interval bounds (t_in and t_out), we could verify subtree membership in O(1) time and restrict our scans strictly to non-tree edges incident to nodes inside subtree A, which would significantly reduce constant-factor overhead on sparse graphs.

4. Key Conceptual Insight Gained

Before diving into this project, I knew very little about tree packing and what made it so effective. I didn't truly grasp why a minimum cut is mathematically guaranteed to cross at least one spanning tree in two or fewer edges. Seeing the implementation firsthand helped me understand the pigeonhole principle. Because every spanning tree must cross a min-cut C at least once, packing k trees distributes the total edge load. If maximum edge loads are balanced, the total number of tree crossings across C cannot exceed 2k, which mathematically forces at least one tree in the packing to cross C at most $\lfloor 2k / k \rfloor = 2$ times. This project helped me change my entire understanding of randomized graph algorithms.

Individual Reflection: Stoer-Wagner Algorithm
Aarav Khemani

1. Worst case

Stoer-Wagner is deterministic and does the same work on every graph of a given size, so its worst case is about graph shape. The matrix version always costs $O(n^3)$ time and $O(n^2)$ memory, so its worst case is a large sparse graph. At n = 1600 with m = 3n it took 4,866 ms while my priority queue version took 427 ms, because the matrix scans 1,600 entries for every pick even though the graph has only 4,800 edges. The priority queue version has the opposite worst case. Its cost is O(nm log n), so on dense graphs it reaches $n^3 \log n$, and at n = 400 with $m = n(n-1)/4$ it is 5.6 times slower than the matrix version. It also has a floor on sparse graphs, because each phase starts by pushing every remaining vertex onto the heap.

2. Hardest design decision

The hardest part was the priority queue version. std::priority_queue cannot change an entry in place, so every time a vertex's key increases I push a fresh (key, vertex) entry, and when popping I skip any entry whose vertex is already in the group or whose key no longer matches key[v]. Merging the last vertex into the one before it in the adjacency maps was the second difficulty. For every neighbour u of the last vertex I add the edge count to both adj[prev][u] and adj[u][prev] and erase the last vertex from adj[u]. I also have to erase the edge between the two merged vertices, otherwise it turns into a self-loop that breaks the next phase.

I checked the result against a brute-force search over every bipartition on 520 random graphs with up to 14 vertices, against known answers (cycles, complete graphs, paths, stars, two clusters), and I compared the two versions against each other on graphs up to n = 150.

3. What I would do differently

If I did this again from scratch, I would first work through the algorithm by hand on paper, before any code exists. I would draw a small graph, write down each vertex's key as it joins, pick s and t, record the cut of the phase, merge, and repeat until one vertex is left. That would give me test cases whose answers I worked out myself, and I would understand the invariant before relying on code that happened to pass. I would also plan my time better. The implementation took far less time than the benchmarks, predictions, report and video, and I treated those as afterthoughts. Next time I would split the weeks up front, write the predictions before running anything, and leave time to rerun experiments.

4. What I did not fully understand before

I did not understand why a single number, key[t], is the whole cut of a phase. When t is added last, every other vertex is already in the group, and key[t] counts the edges from t into the group. That is every edge leaving t, so key[t] is exactly the size of the cut that puts t alone on one side, in any order. After merges it adds up parallel edges as weights. On the triangle 0–1–2 with vertex 3 attached to 2, ending on vertex 3 gives key 1, its one edge, and ending on vertex 2 gives key 3, all three of its edges. This is why a phase needs no extra work to get its cut: in my code it is the single line best = min(best, key[last]). What the maximum adjacency order adds is that this cut is also a minimum s–t cut, which is what makes merging s and t safe.